

---

## BCA EXAMINATION NOTES

Course Code: CA-112

# CA-112 Programming using C++

---

### Exam Pattern:

Q1. Attempt Any Six – 6 marks

Q2. Attempt Any Three – 6 marks

Q3. Attempt Any Two – 6 marks

Q4A. Attempt Any One – 4 marks | Q4B. Compulsory – 2 marks

Q5. Attempt Any One – 6 marks

---

# UNIT 1: Introduction to C++

---

## 1.1 History of C++

C++ was developed by Bjarne Stroustrup at Bell Laboratories (AT&T;) in the early 1980s. It was initially called 'C with Classes'. In 1983, the name was changed to C++. The ++ operator in C means increment, symbolizing that C++ is an enhanced version of C. C++ was standardized by ISO/IEC in 1998 (C++98), with later standards in 2003, 2011 (C++11), 2014, 2017, and 2020.

## 1.2 Advantages of C++

- Object-Oriented: Supports OOP concepts like encapsulation, inheritance, polymorphism.
- High Performance: Compiled language, runs fast, close to hardware.
- Portability: Programs can be compiled and run on different platforms.
- Rich Library: Standard Template Library (STL) provides powerful data structures and algorithms.
- Reusability: Code reuse through classes and inheritance.
- Flexibility: Supports both procedural and object-oriented programming.
- Memory Management: Direct memory manipulation via pointers.
- Extensibility: User-defined types via classes and structures.

## 1.3 Difference between C++ and Old Programming Languages

**Feature:** C (Procedural) vs C++ (OOP)

**Paradigm:** Procedural vs Object-Oriented + Procedural

**Data Security:** No encapsulation vs Encapsulation hides data

**Code Reuse:** Limited (functions) vs Classes, Inheritance

**Function Overloading:** Not supported vs Supported

**Namespace:** Not available vs Available

**Exception Handling:** Not supported vs Supported

**Default Arguments:** Not available vs Available

## 1.4 Character Set

The character set of C++ includes all valid characters that can be used in a C++ program:

- Letters: A-Z, a-z
- Digits: 0-9
- Special Characters: + - \* / = ( ) { } [ ] ; : , . # & | ^ ~ ! % < > ? ' " \
- Whitespace: Space, Tab, Newline, Carriage Return

## 1.5 Tokens, Identifiers, Keywords, Variables

### Tokens

A token is the smallest individual unit in a C++ program. Types of tokens:

- Keywords – Reserved words (e.g., int, float, if, for, class)
- Identifiers – Names given to variables, functions, classes
- Constants – Fixed values (e.g., 10, 3.14, 'A')
- Strings – Sequence of characters in double quotes
- Operators – Symbols performing operations (+, -, \*, /)
- Punctuators – Symbols like ; , { } ( )

## Identifiers

Identifiers are user-defined names for variables, functions, classes, etc.

Rules for Identifiers:

- Must begin with a letter (A-Z, a-z) or underscore (`_`)
- Can contain letters, digits, and underscores
- Cannot be a keyword
- Case-sensitive (Age and age are different)
- No special characters or spaces allowed

## Keywords

Keywords are reserved words with predefined meanings in C++. They cannot be used as identifiers.

Examples:

- `int, float, double, char, bool, void, if, else, for, while, do, switch, case, break, continue, return, class, public, private, protected, new, delete, this, virtual, inline, friend, namespace, template, try, catch, throw`

## Variables

A variable is a named memory location that stores data. Syntax: `data_type variable_name = value;`

Example: `int age = 20; float salary = 25000.50; char grade = 'A';`

Variable must be declared before use. The data type determines what kind of data the variable stores and how much memory it occupies.

## 1.6 Operators

**Arithmetic:** `+` `-` `*` `/` `%` – Addition, Subtraction, Multiply, Divide, Modulus

**Relational:** `==` `!=` `<` `>` `<=` `>=` – Comparison operators

**Logical:** `&&` `||` `!` – AND, OR, NOT

**Assignment:** `=` `+=` `-=` `*=` `/=` `%=` – Assign values

**Increment/Decrement:** `++` `--` – Increase/decrease by 1

**Bitwise:** `&` `|` `^` `~` `<<` `>>` – Bit-level operations

**Conditional/Ternary:** `?:` – condition ? `expr1` : `expr2`

**sizeof:** `sizeof()` – Returns size in bytes

## 1.7 Operator Precedence and Associativity

Operator Precedence determines the order in which operators are evaluated in an expression. Higher precedence operators are evaluated first. Associativity determines the direction of evaluation when operators have the same precedence (left-to-right or right-to-left).

Precedence order (high to low): `() [] :: | ++ -- (unary) | * / % | + - | << >> | < <= > >= | == != | & | ^ | || && | || |`  
`?: | = += -= | ,`

# UNIT 2: Control Structure and Array

---

## 2.1 If Statements

### if Statement

Executes a block of code if the condition is true.

Syntax: `if (condition) { statements; }`

### **if...else Statement**

Executes one block if condition is true, another if false.

Syntax: `if (condition) { ... } else { ... }`

### **if...else if...else**

Tests multiple conditions in sequence.

Syntax: `if (cond1) { ... } else if (cond2) { ... } else { ... }`

### **Nested if**

An if statement inside another if statement. Used when multiple conditions need to be tested hierarchically.

### **Switch Statement**

Used to select one block from multiple alternatives based on the value of a variable/expression.

Syntax: `switch(expression) { case val1: ... break; case val2: ... break; default: ... }`

The break statement exits the switch block. The default case executes when no case matches.

## **2.2 Loops**

### **while Loop**

Repeats a block while the condition is true. Checks condition before executing. Syntax: `while(condition) { body; }`

### **for Loop**

Most commonly used loop with initialization, condition, and update in one line. Syntax: `for(init; condition; update) { body; }`

### **do...while Loop**

Executes the body at least once, then checks condition. Syntax: `do { body; } while(condition);`

### **Nested Loop**

A loop inside another loop. The inner loop completes all iterations for each iteration of the outer loop. Used for 2D patterns, matrices, etc.

## **2.3 Expressions and Qualifiers**

An expression is a combination of operators and operands that evaluates to a value. e.g., `a + b * c`

Qualifiers/Type Modifiers modify the basic data types:

- `const` – Value cannot be changed after initialization
- `volatile` – Value can be changed by external factors
- `signed` – Can store positive and negative values (default)
- `unsigned` – Only stores non-negative values
- `short` – Reduces storage size
- `long` – Increases storage size

## **2.4 Arrays**

An array is a collection of elements of the same data type stored in contiguous memory locations.

Declaration: `data_type array_name[size];` e.g., `int marks[5];`

Accessing elements: `array_name[index]` (index starts from 0)

### Multidimensional Array

An array of arrays. 2D array: `int matrix[rows][cols]`; Accessed as `matrix[i][j]`

Example: `int a[3][3]`; – a 3x3 matrix

## 2.5 Strings and String Functions

A string is an array of characters terminated by a null character ('\0').

Declaration: `char str[20]`; or `char str[] = "Hello"`;

Array of Strings: `char names[5][20]`; – stores 5 strings each of max 20 chars

Important String Functions (from `<string.h>`):

- `strlen(s)` – Returns length of string
- `strcpy(dest, src)` – Copies src to dest
- `strcat(s1, s2)` – Appends s2 to s1
- `strcmp(s1, s2)` – Compares two strings (0 if equal)
- `strupr(s)` – Converts to uppercase
- `strlwr(s)` – Converts to lowercase

## UNIT 3: Array, Structures and Union

---

### 3.1 Structure Declaration and Definition

A structure is a user-defined data type that groups variables of different data types under one name.

Syntax: `struct struct_name { data_type member1; data_type member2; ... };`

Example: `struct Student { int rollno; char name[20]; float marks; };`

Accessing members: Use dot (.) operator: `s.rollno = 101`;

### 3.2 Use of Structure and Union

Structure: All members have their own memory. Total size = sum of all member sizes. Used to group related data (e.g., student record, employee record).

Union: All members share the same memory location. Total size = size of the largest member. Used for memory-efficient storage when only one member is needed at a time.

### 3.3 Difference between Structure and Union

**Memory:** Structure – Each member has separate memory | Union – All members share same memory

**Size:** Structure – Sum of all member sizes | Union – Size of largest member

**Simultaneous Use:** Structure – All members can be used at once | Union – Only one member used at a time

**Keyword:** Structure – `struct` | Union – `union`

### 3.4 Function Components

A function is a self-contained block of code that performs a specific task.

Components of a Function:

- Return Type – Data type of the value returned

- Function Name – Identifier for the function
- Parameters – Input values passed to the function
- Function Body – Block of code (statements)
- Return Statement – Sends value back to caller

### 3.5 Parameter Passing

#### Pass by Value

A copy of the argument is passed to the function. Changes inside function do not affect original. Default method in C++.

#### Pass by Address (Pointer)

Address of the variable is passed. Changes inside function affect original variable. Uses pointer parameter: `void func(int *p)`

#### Pass by Reference

Reference (alias) to the original variable is passed. Changes inside function affect original. Uses `&` in parameter: `void func(int &a;)`

### 3.6 Inline Function

An inline function is expanded at the point of call (like a macro), avoiding function call overhead. Defined with inline keyword.

Example: `inline int square(int x) { return x * x; }`

Advantages: Faster execution (no function call overhead). Disadvantage: Increases code size if function is large.

### 3.7 Recursive Function

A function that calls itself directly or indirectly is called a recursive function. It must have a base case to stop recursion, and a recursive case that calls itself.

Example: `int factorial(int n) { if(n == 0) return 1; return n * factorial(n-1); }`

### 3.8 Friend Function

A friend function is a non-member function that has access to private and protected members of a class. Declared inside the class with keyword friend.

Example: `friend void display(MyClass obj);` – this function is not a member but can access private data of MyClass.

## UNIT 4: Pointers and Preprocessor Directives

---

### 4.1 Introduction to Pointers

A pointer is a variable that stores the memory address of another variable.

Declaration: `data_type *pointer_name;` e.g., `int *p;`

Address operator (`&`): Gives address of a variable. e.g., `p = &x;`

Dereference operator (`*`): Accesses value at the address. e.g., `*p` gives value at address stored in p.

Uses of Pointers: Dynamic memory allocation, passing arrays to functions, implementing linked lists and trees, returning multiple values from functions.

## 4.2 Pointer to Function, Void Pointer

### Pointer to Function

A pointer that stores the address of a function. Syntax: `return_type (*ptr_name)(param_types);` Allows calling functions indirectly.

### Void Pointer

A pointer of type `void*` that can hold the address of any data type. It is a generic pointer. Must be cast to appropriate type before dereferencing.

Example: `void *ptr; int x = 10; ptr = &x; cout << *(int*)ptr;`

## 4.3 Memory Management Operators

C++ uses `new` and `delete` operators for dynamic memory allocation.

- `new` – Allocates memory at runtime on heap. e.g., `int *p = new int;`
- `delete` – Frees allocated memory. e.g., `delete p;`
- `new[]` – Allocates array. e.g., `int *arr = new int[10];`
- `delete[]` – Frees array memory. e.g., `delete[] arr;`

Object to Pointer: `MyClass *ptr = new MyClass();` Accessing members: `ptr->member;`

## 4.4 Preprocessor Directives

Preprocessor directives are instructions processed before compilation. They begin with `#` symbol.

### **#define**

Defines a macro (symbolic constant). e.g., `#define PI 3.14159` or function-like macro: `#define MAX(a,b) ((a)>(b)?(a):(b))`

### **#include**

Includes header files. e.g., `#include <iostream>` (system header) or `#include "myfile.h"` (user header)

### **#error**

Generates a compiler error with a custom message. e.g., `#error "This compiler is not supported"`

## DEFINITIONS (2-Mark Ready)

---

**Token:** A token is the smallest meaningful unit in a C++ program, including keywords, identifiers, literals, operators, and punctuators.

**Identifier:** An identifier is a user-defined name given to variables, functions, classes, or other entities in a program.

**Keyword:** A keyword is a reserved word in C++ with a predefined meaning that cannot be used as an identifier. e.g., `int`, `if`, `class`.

**Array:** An array is a collection of elements of the same data type stored in consecutive memory locations, accessed using an index.

**Pointer:** A pointer is a variable that stores the memory address of another variable of a specific data type.

**Structure:** A structure is a user-defined data type that groups together variables of different data types under a single name.

**Inline Function:** An inline function is a function whose code is inserted directly at the point of call, eliminating function call overhead.

**Recursive Function:** A recursive function is one that calls itself, directly or indirectly, to solve a problem by reducing it to simpler sub-problems.

**Macro:** A macro is a name defined using `#define` that is replaced by its value or code before compilation by the preprocessor.

**Friend Function:** A friend function is a non-member function that has been granted access to the private and protected members of a class.

## 20 MOST IMPORTANT QUESTIONS WITH ANSWERS

---

### Q1. What is C++? State its advantages. [2M]

C++ is a general-purpose, object-oriented programming language developed by Bjarne Stroustrup in 1983. Advantages: (1) Supports OOP concepts, (2) High performance due to compilation, (3) Rich STL library, (4) Supports both procedural and OOP style, (5) Direct memory manipulation via pointers.

### Q2. What are tokens in C++? Name its types. [2M]

Tokens are the smallest individual units recognized by the C++ compiler. Types: (1) Keywords – e.g., `int`, `for`, `class` (2) Identifiers – user-defined names (3) Constants/Literals – fixed values (4) Strings – character sequences in `" "` (5) Operators – `+`, `-`, `*`, `/` (6) Punctuators – `;`, `{}`, `()`

### Q3. Explain if-else if ladder with syntax and example. [4M]

The if-else if ladder allows testing multiple conditions.

Syntax:

```
if(cond1) { ... }  
else if(cond2) { ... }  
else if(cond3) { ... }  
else { ... }
```

Example: To find grade based on marks:

```
if(marks>=90) cout<<"A";  
else if(marks>=75) cout<<"B";  
else if(marks>=60) cout<<"C";  
else cout<<"F";
```

Each condition is checked in order. The first true condition executes.

### Q4. Explain switch statement with syntax. [4M]

Switch statement selects one block of code from multiple cases based on the value of an expression.

Syntax:

```
switch(expression) {  
case val1: statement1; break;  
case val2: statement2; break;  
default: default_statement;
```



}

Rules: (1) Expression must be integer or character type. (2) break prevents fall-through to next case. (3) default executes when no case matches. (4) Case values must be unique constants.

#### **Q5. Write a note on for loop, while loop and do-while loop. [6M]**

FOR LOOP: Used when number of iterations is known.

Syntax: for(init; condition; update) { body; }

Example: for(int i=1; i<=5; i++) cout<

WHILE LOOP: Used when number of iterations is unknown; condition checked before execution.

Syntax: while(condition) { body; }

Example: while(n>0) { sum+=n; n--; }

DO-WHILE LOOP: Body executes at least once; condition checked after execution.

Syntax: do { body; } while(condition);

Example: do { cout<

Comparison: for is used for definite loops; while for indefinite; do-while when body must run at least once.

#### **Q6. What is an array? How to declare and access arrays? [4M]**

An array is a collection of elements of the same data type stored in contiguous memory locations.

Declaration: data\_type array\_name[size];

Example: int marks[5];

Initialization: int marks[5] = {90, 85, 78, 92, 88};

Accessing elements: array\_name[index] where index starts from 0.

Example: marks[0] gives first element (90), marks[4] gives last element (88).

2D Array: int matrix[3][3]; accessed as matrix[i][j].

#### **Q7. Explain multidimensional arrays with example. [4M]**

A multidimensional array is an array with more than one dimension. The most common is the 2D array (matrix).

Declaration: data\_type name[rows][cols];

Example: int a[3][3]; – creates a 3x3 matrix.

Initialization:

int a[2][3] = {{1,2,3},{4,5,6}};

Accessing: a[i][j] where i=row index, j=column index.

Memory: Elements stored row-wise in contiguous locations.

Use: Representing matrices, tables, grids.

#### **Q8. Explain string functions with examples. [4M]**

String functions from <string.h>:

1. strlen(s): Returns number of characters in s excluding null.

Example: strlen("Hello") = 5

2. strcpy(dest, src): Copies src into dest.

Example: strcpy(s2, s1) copies s1 to s2.

3. strcat(s1, s2): Appends s2 to end of s1.

Example: strcat("Hello ", "World") = "Hello World"

4. strcmp(s1, s2): Compares strings. Returns 0 if equal, positive/negative if not.

5.strupr(s): Converts to uppercase.

6. strlwr(s): Converts to lowercase.

**Q9. Define structure. Write difference between structure and union. [6M]**

STRUCTURE: A structure is a user-defined data type that groups variables of different types.

Syntax: struct Student { int roll; char name[20]; float marks; };

Accessing: Student s; s.roll = 101;

UNION: A union is a user-defined type where all members share the same memory location.

Syntax: union Data { int i; float f; char c; };

Differences:

1. Memory: Structure gives each member separate memory; union shares one location.
2. Size: Structure size = sum of all members; Union size = largest member.
3. Access: All structure members can be used simultaneously; union only one at a time.
4. Purpose: Structure for grouped records; union for memory efficiency.
5. Keyword: struct vs union.

**Q10. Explain parameter passing methods. [6M]**

There are three methods of passing parameters:

1. PASS BY VALUE: A copy of the argument is sent to the function. Changes to parameter do not affect original variable.

void add(int a, int b) { a = a + b; } // a is local copy

2. PASS BY ADDRESS (POINTER): The address (memory location) of variable is passed. Changes through pointer affect original variable.

void swap(int \*a, int \*b) { int t=\*a; \*a=\*b; \*b=t; }

Call: swap(&x;, &y;);

3. PASS BY REFERENCE: A reference (alias) to the original variable is passed. Changes affect original without using pointers.

void swap(int &a;, int &b;) { int t=a; a=b; b=t; }

Call: swap(x, y);

Comparison: Pass by value is safest; by reference is most convenient; by address gives direct memory control.

**Q11. What is an inline function? Write its advantages and disadvantages. [4M]**

Inline function is a function whose body is substituted at the point of call during compilation, instead of a regular function call.

Syntax: inline int square(int x) { return x\*x; }

Advantages:

1. Eliminates function call overhead (no stack push/pop)
2. Faster execution for small functions
3. Can be used as type-safe alternative to macros

Disadvantages:

1. Increases code size (code duplication)
2. Not suitable for large or recursive functions
3. Compiler may ignore inline for complex functions

**Q12. What is a recursive function? Explain with factorial example. [4M]**

A recursive function is a function that calls itself to solve a problem by breaking it into smaller sub-problems.

Base case: Stops the recursion.

Recursive case: Calls itself with a smaller input.

Example – Factorial:

```
int factorial(int n) {  
    if(n == 0 || n == 1) return 1; // base case  
    return n * factorial(n-1); // recursive case  
}
```

Working of factorial(4):

```
factorial(4) = 4 * factorial(3)  
             = 4 * 3 * factorial(2)  
             = 4 * 3 * 2 * factorial(1)  
             = 4 * 3 * 2 * 1 = 24
```

### **Q13. What is a pointer? Explain its uses. [4M]**

A pointer is a variable that stores the memory address of another variable.

Declaration: `int *p; // p is pointer to int`

Assignment: `int x=10; p=&x; // p stores address of x`

Dereferencing: `*p` gives value at address (10)

Uses of Pointers:

1. Dynamic memory allocation (new, delete)
2. Passing arrays to functions efficiently
3. Implementing data structures (linked list, tree)
4. Returning multiple values from functions
5. Pointer to functions for callbacks
6. Direct memory manipulation

### **Q14. Explain void pointer and pointer to function. [4M]**

VOID POINTER: A void pointer (`void*`) is a generic pointer that can hold address of any data type. It cannot be dereferenced directly – must be cast first.

```
void *ptr; int x=10; ptr=&x; cout<<*(int*)ptr;
```

Use: Used in generic functions like `malloc()` in C.

POINTER TO FUNCTION: A pointer that stores the address of a function.

Syntax: `return_type (*ptr)(param_types);`

Example:

```
int add(int a, int b) { return a+b; }
```

```
int (*fp)(int,int) = add;
```

cout<<Use: Callbacks, function arrays, runtime polymorphism.

### **Q15. Explain memory management operators new and delete. [4M]**

C++ provides new and delete for dynamic (heap) memory management.

new operator: Allocates memory at runtime.

Syntax: `pointer = new data_type;`

Example: `int *p = new int; *p = 10;`

For array: `int *arr = new int[10];`

For object: `MyClass *obj = new MyClass();`

delete operator: Releases dynamically allocated memory.

Syntax: `delete pointer;`

Example: `delete p; // free single variable`

For array: `delete[] arr; // free array`

For object: `delete obj;`

Important: Always delete what you new to avoid memory leaks.

#### **Q16. What are preprocessor directives? Explain #define, #include, #error. [6M]**

Preprocessor directives are instructions processed before actual compilation. They start with #.

1. #define: Defines a macro (symbolic constant or function-like).

Syntax: `#define NAME value`

Example: `#define PI 3.14`

Function macro: `#define AREA(r) (PI*r*r)`

Advantage: Avoids magic numbers, easy to change at one place.

2. #include: Includes a header file into the program.

System header: `#include <iostream>`

User header: `#include "mylib.h"`

Provides access to standard library functions.

3. #error: Forces a compiler error with a custom message.

Example: `#error "Header not supported on this platform"`

Used in conditional compilation to catch invalid configurations.

Other directives: `#ifdef`, `#ifndef`, `#endif`, `#pragma`, `#undef`.

#### **Q17. Explain operators in C++ with examples. [6M]**

C++ supports many types of operators:

1. Arithmetic: `+` `-` `*` `/` `%` — Used for mathematical calculations.

`5+3=8`, `10%3=1`

2. Relational: `==` `!=` `<` `>` `<=` `>=` — Compares values, returns true/false.

`5>3` is true, `5==6` is false

3. Logical: `&&` (AND) `||` (OR) `!` (NOT) — Combines boolean expressions.

`(5>3)&&(4<6)` is true

4. Assignment: `=` `+=` `-=` `*=` `/=` `%=` — Assign values.

`a+=5` means `a=a+5`

5. Increment/Decrement: `++` `--` — Increase or decrease value by 1.

Pre: `++a` (increment first, then use); Post: `a++` (use first, then increment)

6. Bitwise: `&` `|` `^` `~` `<<` `>>` — Operate on bits.

$5\&3=1$ ,  $5|3=7$ ,  $5<<1=10$

7. Conditional (Ternary): condition ? expr1 : expr2  
max = (a>b) ? a : b;

**Q18. Explain rules for naming identifiers. [2M]**

Rules for naming identifiers in C++:

1. Must begin with a letter (A-Z, a-z) or underscore (`_`)
2. Can contain letters, digits (0-9), and underscores
3. Cannot be a C++ keyword (int, float, class, etc.)
4. Case-sensitive: 'Name' and 'name' are different
5. No spaces or special characters allowed
6. No length limit (but practical limit: 255 chars)

Valid: myVar, \_count, student1

Invalid: 1num (starts with digit), my-var (hyphen), int (keyword)

**Q19. What is operator precedence and associativity? [2M]**

Operator Precedence: Defines the priority of operators when multiple operators appear in an expression. Higher precedence operator is evaluated first.

Example:  $2+3*4 = 2+12 = 14$  (because \* has higher precedence than +)

Associativity: Defines the direction of evaluation when operators have the same precedence.

Left-to-right: Most operators (arithmetic, logical)

Right-to-left: Assignment, unary, ternary

Example:  $a=b=c=5$  — evaluated as  $a=(b=(c=5))$  (right to left)

**Q20. What is the difference between while loop and do-while loop? [2M]**

While Loop:

1. Condition is checked BEFORE executing the body
2. Body may not execute even once if condition is false initially
3. Entry-controlled loop

Syntax: while(condition) { body; }

do-while Loop:

1. Condition is checked AFTER executing the body
2. Body always executes at least ONCE
3. Exit-controlled loop

Syntax: do { body; } while(condition);

Key difference: do-while guarantees minimum one execution, while does not.